TOP 50 PYTHON INTERVIEW QUESTIONS

→ PAGE 1 ←

① What is Python?

Ans: Python is a high-level, interpreted, general-purpose programming language. It was created by Guido van Rossum and first released in 1991. Python is known for its simple syntax and readability.

② What are the key features of Python?

Ans:
- Simple and easy to learn.
- High-level language.
- Interpreted and dynamically typed.
- Platform independent.
- Extensive standard library.
- Supports multiple programming paradigms (OOP, Functional, etc.)
- Open source and large community support.

Example:
```
print("Hello, Python!")
# Output: Hello, Python!
```

③ Is Python an interpreted or compiled language?

Ans: Python is an interpreted language. The Python code is executed line-by-line by the interpreter at runtime. It does not need a separate compilation step.

Example:
```
x = 10
y = 20
print(x + y)
# Output: 30
```

④ What is the difference between mutable and immutable objects in Python?

Ans: Mutable objects can be changed after they are created. Immutable objects cannot be changed once created.

Mutable Examples: list, dict, set, bytearray

Immutable Examples: int, float, str, tuple, bool

Example:
```
a = [1,2,3]     # Mutable
a[0] = 10       # allowed
b = (1,2,3)     # Immutable
b[0] = 10       # Error
```

⑤ What is the difference between List and Tuple?

List	Tuple
• List is mutable (can be changed).	• Tuple is immutable (cannot be changed).
• Defined using square brackets [.].	• Defined using parentheses (.).
• Slower than tuples.	• Faster than lists.
• Uses more memory.	• Uses less memory.
• Example: my_list = [1, 2, 3]	• Example: my_tuple = (1, 2, 3)

Example:
```
L = [1,2,3]
L[0] = 10      # Valid
T = (1,2,3)
T[0] = 10      # TypeError
```

* Next Page → Questions 6 to 10

(6) What is the difference between Dictionary and Set in Python?

Ans:

Dictionary	Set
• Stores data in key : value pairs.	• Stores only unique values.
• Keys must be unique.	• No key : value concept.
• Ordered (from Python 3.7+).	• Unordered (no index).
• Mutable.	• Mutable.
• Defined using curly braces {}.	• Defined using curly braces or set().
• Example: {'a': 1, 'b': 2}	• Example: {1, 2, 3} or set([1,2,3])

Example:

```
d = {'a': 1, 'b': 2}
# Dictionary
s = {1, 2, 3}
# Set
```

(7) What is List Comprehension in Python?

Ans: List comprehension provides a concise way to create lists. It consists of brackets containing an expression followed by a for clause, and optionally, an if clause.

Example:

```
# Create a list of squares of numbers from 1 to 5
squares = [x*x for x in range(1, 6)]
print(squares)
# Output: [1, 4, 9, 16, 25]
```

(8) What is tuple packing and unpacking in Python?

Ans: Packing: Assigning multiple values to a single tuple.
Unpacking: Extracting values from a tuple and assigning them to multiple variables.

Example:

```
# Packing
t = 1, 2, 3          # t is (1, 2, 3)
# Unpacking
a, b, c = t          # a=1, b=2, c=3
```

(9) What is the difference between Shallow Copy and Deep Copy?

Ans:

Shallow Copy	Deep Copy
• Creates a new object but inserts references to the original objects.	• Creates a new object and recursively copies all nested objects.
• Changes in nested objects reflect in both.	• Changes in nested objects do not reflect in original.
• Uses less memory.	• Uses more memory.
• Created using copy.copy() or slicing.	• Created using copy.deepcopy().

Example:

```
import copy
l1 = [1, [2, 3]]
l2 = copy.copy(l1)
l3 = copy.deepcopy(l1)
l1[1][0] = 99
print(l2)    # [1, [99, 3]]
print(l3)    # [1, [2, 3]]
```

(10) What is the difference between *args and **kwargs in Python?

Ans:

*args (Non-keyword Arguments)	**kwargs (Keyword Arguments)
• Used to pass variable number of non-keyword arguments.	• Used to pass variable number of keyword arguments.
• Treated as a tuple.	• Treated as a dictionary.
• Defined with a single *.	• Defined with double **.

• Example:
`def func(*args):`
`  print(args)  # (1, 2, 3)`
`func(1, 2, 3)`
	• Example:
	`def func(**kwargs):`
	`  print(kwargs)  # {'a':1, 'b':2}`
	`func(a=1, b=2)`

Example:

```
def func(*args, **kwargs):
    print("args:", args)
    print("kwargs:", kwargs)
func(1, 2, 3, a=10, b=20)
# Output:
args: (1, 2, 3)
kwargs: {'a': 10, 'b': 20}
```

(11) What is a Lambda function in Python ?

Ans: A lambda function is a small anonymous function defined using the lambda keyword. It can have any number of arguments but only one expression.

Example:

```
add = lambda a, b: a + b
print(add(3, 4))    # Output: 7
# Used for short, simple operations.
```

(12) What are map(), filter() and reduce() functions in Python ?

Ans:

- map(): Applies a function to all items in an iterable and returns a map object.
- filter(): Filters elements based on a function that returns True and returns a filter object.
- reduce(): Applies a rolling computation to sequential pairs of values in an iterable and returns a single value.

Example:

```
from functools import reduce
nums = [1, 2, 3, 4]
print(list(map(lambda x: x * 2, nums)))     # [2, 4, 6, 8]
print(list(filter(lambda x: x % 2 == 0, nums)))  # [2, 4]
print(reduce(lambda x, y: x + y, nums))     # 10
```

(13) What is the difference between Local and Global variables ?

Ans:

- Local variable: Declared inside a function and accessible only within that function.
- Global variable: Declared outside any function and accessible throughout the program.

Example:

```
x = 10        # Global variable
def func():
    y = 5     # Local variable
    print(x, y)
# Output: 10 5
```

(14) What is LEGB rule in Python ?

Ans: LEGB stands for Local, Enclosing, Global, Built-in. It is the order in which Python looks for a variable name when it is used in a program.
Python searches in this order:
Local → Enclosing → Global → Built-in

Example:

```
x = "global"
def outer():
    x = "enclosing"
    def inner():
        x = "local"
        print(x)
    inner()
outer()       # Output: local
```

(15) What is Recursion in Python ?

Ans: Recursion is the process of defining a function that calls itself to solve a problem.
It is used when a problem can be divided into similar subproblems.

Example:

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)
print(factorial(5))     # Output: 120
# factorial(5) = 5*4*3*2*1
```

(16) What is a Class and Object in Python ?

Ans: A class is a blueprint or template for creating objects.
An object is an instance of a class.
It contains data (attributes) and behavior (methods).

Example:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

p1 = Person("Aman", 25)
print(p1.name, p1.age)  # Output: Aman 25
```

(17) What is Inheritance in Python ?

Ans: Inheritance is a way to create a new class using the properties and methods of an existing class.
It promotes code reusability.

Example:

```
lass Parent:
    def show(self):
        print("This is parent class")

class Child(Parent):
    pass

c = Child()
c.show()    # Output: This is parent class
```

(18) What is Polymorphism in Python ?

Ans: Polymorphism means "many forms".
It allows methods to do different things based on the object that calls them.
In Python, it is achieved through method overriding and duck typing.

Example:

```
class Animal:
    def sound(self):
        pass
class Dog(Animal):
    def sound(self):
        print("Bark")
class Cat(Animal):
    def sound(self):
        print("Meow")
for a in (Dog(), Cat()):
    a.sound()    # Output: Bark
                          Meow
```

(19) What is Encapsulation in Python ?

Ans: Encapsulation is the concept of wrapping data and methods together in a single unit (class) and restricting direct access to some of the object's components.
In Python, we use _ (single underscore) or __ (double underscore) to indicate private members.

Example:

```
class Bank:
    def __init__(self, balance):
        self.__balance = balance

    def get_balance(self):
        return self.__balance

b = Bank(1000)
print(b.get_balance())    # Output: 1000
# b.__balance  (Not recommended to access directly)
```

(20) What is Abstraction in Python ?

Ans: Abstraction means hiding the implementation details and showing only the essential features to the user.
In Python, it is achieved using abstract classes and abstract methods (using abc module).

Example:

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
class Circle(Shape):
    def area(self):
        return 3.14 * 5 * 5
c = Circle()
print(c.area())    # Output: 78.5
```

(21) What is Method Overloading and Overriding in Python?

Ans:

- Overloading: Same method name with different parameters.
  Python does not support method overloading directly.
- Overriding: Child class provides its own implementation of a method already defined in the parent class.

Example (Overriding):

```
class Parent:
    def show(self):
        print("This is parent")
class Child(Parent):
    def show(self):
        print("This is child")
c = Child()
c.show()        # Output: This is child
```

(22) What is the difference between Static Method, Class Method and Instance Method in Python?

Ans:

- Instance Method: Works on instance and can access instance variables.
- Class Method: Works on class and can access class variables.
- Static Method: Works like a normal function but belongs to the class.

Example:

```
class MyClass:
    class_var = 10
    def instance_method(self):
        print(self.class_var)
    @classmethod
    def class_method(cls):
        print(cls.class_var)
    @staticmethod
    def static_method():
        print("Hello Static Method")
```

(23) What are Magic (Dunder) Methods in Python?

Ans: Magic methods have double underscores before and after the method name.
They are used to define the behavior of objects for built-in operations.
Example: `__init__()`, `__str__()`, `__len__()`, `__add__()`, `__eq__()`, etc.

Example:

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __str__(self):
        return f"Point({self.x}, {self.y})"
p = Point(2, 3)
print(p)      # Output: Point(2, 3)
```

(24) What is @property decorator in Python?

Ans: @property decorator is used to define a getter method.
It allows us to access methods like attributes.
It makes the code more readable and secure.

Example:

```
class Employee:
    def __init__(self, salary):
        self._salary = salary
    @property
    def salary(self):
        return self._salary
e = Employee(50000)
print(e.salary)      # Output: 50000
```

(25) What is Multiple Inheritance and MRO in Python?

Ans:

- Multiple Inheritance: A class inherits from more than one parent class.
- MRO (Method Resolution Order): The order in which Python looks for methods in a class hierarchy.
- Python uses C3 Linearization Algorithm for MRO.

Example:

```
class A:
    def show(self):
        print("A")
class B(A):
    def show(self):
        print("B")
class C(A):
    def show(self):
        print("C")
class D(B, C):
    pass
d = D()
d.show()          # Output: B
print(D.mro())    # Output: [D, B, C, A, object]
```

(26) What is the purpose of try, except, else and finally in Python ?

Ans:

- try : Contains code that might raise an exception.
- except : Handles the exception if it occurs.
- else : Executes if no exception occurs in try block.
- finally : Executes always, whether exception occurs or not. Used for clean-up actions.

Example:

```
try:
    x = 10 / 2
    print(x)
except ZeroDivisionError:
    print("Cannot divide by zero")
else:
    print("No exception occurred")
finally:
    print("This will always execute")
```

(27) What is the use of 'raise' keyword in Python ?

Ans:

The "raise" keyword is used to raise an exception manually. It allows us to trigger a specific error when a particular condition occurs.

Example:

```
def check_age(age):
    if age < 18:
        raise ValueError("Age must be 18 or above")
check_age(15)      # Raises ValueError
```

(28) How can we create Custom Exceptions in Python ?

Ans:

We can create custom exceptions by creating a new class that inherits from the built-in Exception class.

Example:

```
class InsufficientBalanceError(Exception):
    pass
def withdraw(amount, balance):
    if amount > balance:
        raise InsufficientBalanceError("Low balance!")
withdraw(1000, 500)    # Raises custom exception
```

(29) How does File Handling work in Python ?

Ans:

File handling allows us to open, read, write, update and close files.
Python provides built-in functions like open(), read(), write(), close(), etc.

Example:

```
# Writing to a file
f = open("demo.txt", "w")
f.write("Hello Python!")
f.close()
# Reading from a file
f = open("demo.txt", "r")
data = f.read()
print(data)      # Output: Hello Python!
f.close()
```

(30) What is the use of 'with' statement in Python ?

Ans: The "with" statement is used with context managers. It automatically handles resource allocation and deallocation (like opening and closing a file). It ensures proper cleanup even if an exception occurs.

Example:

```
with open("demo.txt", "r") as file:
    data = file.read()
    print(data)
# File is automatically closed after the block
# No need of file.close()
```

(31) What is an Iterator in Python ?

Ans: An iterator is an object that contains a countable number of values. It implements the `__iter__()` and `__next__()` methods. It is used to iterate over a sequence (list, tuple, string, etc.).

Example:

```
my_list = [10, 20, 30]
it = iter(my_list)
print(next(it))      # 10
print(next(it))      # 20
print(next(it))      # 30
# StopIteration (no more items)
```

(32) What is a Generator in Python ?

Ans: A generator is a special type of iterator which is generated using a function and the yield keyword. Generators are memory efficient as they generate values one at a time.

Example:

```
def count(n):
    i = 1
    while i <= n:
        yield i
        i += 1

for num in count(3):
    print(num)  # Output: 1 2 3
```

(33) What is the difference between "yield" and "return" ?

Ans:

yield	return
• Used in generator functions.	• Used in normal functions.
• Pauses the function and returns a value.	• Terminates the function and returns a value.
• Saves the state of the function.	• Does not save the state.
• Can return multiple values.	• Returns only one value.

Example:

```
# yield
def gen():
    yield 1
    yield 2
# Output: 1, 2

# return
def func():
    return 1
    return 2  # Unreachable
# Output: 1
```

(34) What is a Decorator in Python ?

Ans: A decorator is a function that takes another function as an argument and extends the behavior of the original function without modifying its code.

Example:

```
def my_decorator(func):
    def wrapper():
        print("Before function call")
        func()
        print("After function call")
    return wrapper
@my_decorator
def say_hello():
    print("Hello Python!")
say_hello()
# Output: Before function call
#         Hello Python!
#         After function call
```

(35) What is a Closure in Python ?

Ans: A closure is a function object that remembers values in the enclosing scope even if the outer function has finished execution.

Example:

```
def outer(x):
    def inner(y):
        return x + y
    return inner

add_five = outer(5)
print(add_five(10))   # Output: 15
# inner() remembers x = 5
```

(36) What is Python Memory Management?

Ans: Python manages memory automatically using a combination of private heap space and the garbage collector.
It handles allocation, deallocation and reclaims memory that is no longer in use.

Example:

```
import sys
a = [1, 2, 3]
print(sys.getsizeof(a))  # size of object
del a    # remove reference
# Memory will be freed by garbage collector
  when not referenced.
```

(37) What is Garbage Collection in Python?

Ans: Garbage Collection is the process of reclaiming memory occupied by objects that are no longer in use.
Python uses reference counting along with a cyclic garbage collector to free memory.

Example:

```
import gc
a = [ ]
b = { }
a.append(b)
b["key"] = a
del a, b
gc.collect()  # manually trigger GC
print("Garbage Collected")
```

(38) What is GIL (Global Interpreter Lock) in Python?

Ans: GIL is a mutex that allows only one thread to execute Python bytecode at a time.
It simplifies memory management and makes Python threads safe, but limits true parallelism in CPU-bound tasks.

Example / Note:

- GIL affects multithreading in Python.
- Only one thread runs Python bytecode at a time even on multi-core CPUs.
- I/O-bound tasks can still benefit from threads.

(39) What is the difference between Multithreading and Multiprocessing in Python?

Ans:

Multithreading	Multiprocessing
• Uses threads within the same process.	• Uses multiple processes.
• Shares same memory space.	• Does not share memory.
• Lightweight and faster to create.	• Heavyweight and slower to start.
• Limited by GIL (not ideal for CPU-bound tasks).	• True parallelism (no GIL issue).
• Ideal for I/O-bound tasks.	• Suitable for CPU-bound tasks.

Example:

```
# Multithreading
import threading
def task():
    print("Thread running")
t = threading.Thread(target=task)
t.start()

# Multiprocessing
from multiprocessing import Process
def task():
    print("Process running")
p = Process(target=task)
p.start()
```

(40) What is Async Programming in Python (async/await)?

Ans: Async programming allows writing concurrent code using async and await keywords.
It is based on event loop and is useful for I/O-bound and high concurrency applications.

Example:

```
import asyncio
async def hello():
    print("Hello")
    await asyncio.sleep(1)
    print("World")

async def main():
    await hello()

asyncio.run(main())
# Output:
# Hello
# World
```

(41) What is List Comprehension in Python ?

Ans: List Comprehension provides a concise way to create lists. It consists of brackets [ ] containing an expression followed by a for clause, and zero or more for or if clauses.

Example :

```
# Square of numbers 0 to 4
squares = [x*x for x in range(5)]
print(squares)     # Output: [0, 1, 4, 9, 16]

# With condition
evens = [x for x in range(10) if x % 2 == 0]
print(evens)       # Output: [0, 2, 4, 6, 8]
```

(42) What is the difference between 'deepcopy' and 'shallow copy' in Python ?

Ans:
- Shallow copy copies the references of objects. Changes in nested objects affect both copies.
- Deep copy copies the actual objects recursively. Changes in nested objects do not affect the original.

Example :

```
import copy
a = [1, [2, 3], 4]
b = a.copy()            # Shallow copy
c = copy.deepcopy(a)    # Deep copy

b[1][0] = 99
c[1][0] = 77
print(a)    # Output: [1, [99, 3], 4]
print(b)    # Output: [1, [99, 3], 4]
print(c)    # Output: [1, [77, 3], 4]
```

(43) What is a Lambda Function in Python ?

Ans: Lambda functions are small anonymous functions defined using the "lambda" keyword. They can have any number of arguments but only one expression.
They are often used with functions like map(), filter(), and reduce().

Example :

```
add = lambda a, b: a + b
print(add(3, 4))  # Output: 7

# With map()
nums = [1, 2, 3, 4]
squares = list(map(lambda x: x*x, nums))
print(squares)    # Output: [1, 4, 9, 16]
```

(44) What is '*args' and '**kwargs' in Python ?

Ans: *args allows a function to accept any number of positional arguments.
**kwargs allows a function to accept any number of keyword arguments.
They provide flexibility when we don't know the number of arguments in advance.

Example :

```
def my_func(*args, **kwargs):
    print("args:", args)
    print("kwargs:", kwargs)

my_func(1, 2, 3, name="Aman", age=25)
# Output:
# args: (1, 2, 3)
# kwargs: {'name': 'Aman', 'age': 25}
```

(45) What is a Decorator with Arguments in Python ?

Ans: Decorators with arguments are functions that return a decorator. They allow us to pass parameters to the decorator and modify the behaviour of the decorated function dynamically.

Example :

```
def repeat(times):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(times):
                func(*args, **kwargs)
        return wrapper
    return decorator

@repeat(3)
def say_hello(name):
    print("Hello", name)
say_hello("Python")
# Output:
# Hello Python
# Hello Python
# Hello Python
```